ANDROID THEORY – QUESTION & MODEL ANSWERS

==========

Q1. (5 MARKS)

What are the various files used for developing an Android application in Android Studio?

==========

MODEL ANSWER:

The important files used for developing an Android application in Android Studio are:

1. AndroidManifest.xml

   It contains important information about the application such as

   application components, permissions, activities, services, etc.

2. Java/Kotlin Files

   These files contain the programming logic of the Android application.

3. XML Layout Files

   These files are used to design the user interface of the application.

4. Gradle Files

   Gradle files contain build configuration and dependency information.

   They are used to build and manage the Android project.

5. Resource Files

   These include images, strings, colors, dimensions and other resources

   required by the application.

Thus, these files work together to develop, configure and build an Android application in Android Studio.

==========

Q2. (10 MARKS)

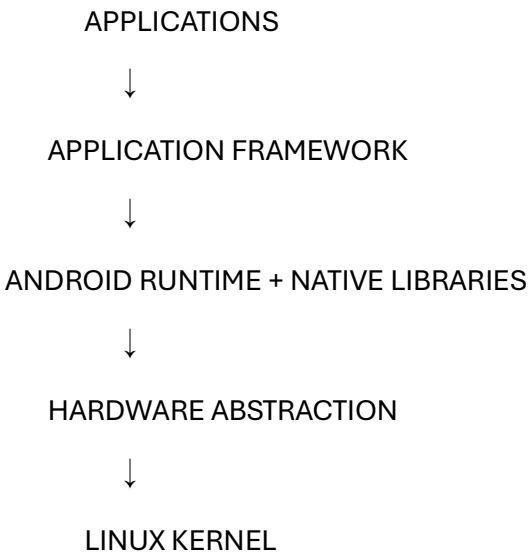
Draw and explain the Android System Architecture.

==========

MODEL ANSWER:

Android system architecture is organized into different layers.

The major layers are:



1. APPLICATIONS

This is the top layer of Android architecture.

It contains applications used by the users such as:

- Phone
- Messages
- Camera
- Contacts
- Browser
- Third-party applications

These applications use the services provided by the lower layers.

2. APPLICATION FRAMEWORK

The Application Framework provides important services that help developers create Android applications.

Important components include:

- Activity Manager
- Window Manager
- Notification Manager
- Content Providers
- Package Manager
- Resource Manager

For example, Activity Manager manages the lifecycle of activities.

3. ANDROID RUNTIME

Android Runtime provides the environment required for executing Android applications.

It includes:

- Core libraries
- ART (Android Runtime)

ART executes application code and provides memory management, garbage collection and other runtime services.

4. NATIVE LIBRARIES

Android provides several native libraries that support important application functions.

Examples include libraries for:

- Graphics
- Media
- Database
- Web browsing
- 2D and 3D graphics

These libraries provide functionality used by Android applications.

5. HARDWARE ABSTRACTION LAYER (HAL)

HAL provides standard interfaces between Android's higher-level software and hardware components.

It allows Android to communicate with hardware such as:

- Camera
- Bluetooth
- Sensors
- Audio hardware

6. LINUX KERNEL

The Linux Kernel is the bottom layer of Android architecture.

It provides:

- Device drivers
- Memory management
- Process management
- Power management
- Security
- Network management

Thus, the Linux Kernel communicates with hardware and provides essential services to the upper layers.

CONCLUSION:

Android architecture uses a layered structure where applications use framework services, runtime and libraries, which ultimately communicate with hardware through HAL and the Linux Kernel.

==========

Q3. (5 MARKS)

Define ART and what is its role in applications?

==========

MODEL ANSWER:

ART stands for Android Runtime.

It is the runtime environment used by Android to execute applications.

ROLE OF ART:

1. It executes Android application code.

2. It converts application bytecode into machine-level code for execution.

3. It provides memory management.

4. It performs garbage collection.

5. It improves application performance and efficiency.

Therefore, ART provides the environment required for Android applications to run efficiently.

==========

Q4. (5 MARKS)

What is Dexing?

==========

MODEL ANSWER:

Dexing is the process of converting compiled application code into DEX (Dalvik Executable) format.

DEX files are designed for execution on Android Runtime.

The basic process is:

Java/Kotlin Code

↓

Compiled Code

↓

DEX File

↓

Android Runtime

↓

Application Execution

DEX format allows Android applications to package and execute their compiled code efficiently.

==========

Q5. (10 MARKS)

What are the two different extensions used for Android application executables? Also bring the differences between these two.

==========

MODEL ANSWER:

The two executable extensions are:

1. .dex
2. .odex

1. .DEX

DEX stands for Dalvik Executable.

A .dex file contains compiled Android application bytecode.

It is the standard executable format used by Android applications.

Features:

- Contains application bytecode.
- Designed for Android execution.
- Can be packaged inside an APK.
- Used by the Android runtime.

2. .ODEX

ODEX stands for Optimized DEX.

It is an optimized form of DEX code that can be used to improve application loading and execution.

Features:

- Contains optimized application code.
- Used for optimization.
- Can reduce some work required during application startup.
- Associated with Android system/application optimization.

DIFFERENCES:

	.DEX	.ODEX
Meaning	Dalvik Executable	Optimized Dalvik Executable

Purpose	Contains compiled bytecode	Contains optimized code
Optimization	Standard format	Optimized format
Usage	General Android application code	Optimization of execution

CONCLUSION:

DEX is the standard compiled Android executable format, whereas ODEX represents optimized executable code.

==========

Q6. (5 MARKS)

What is AAPT? Bring its importance in Android Programming.

==========

MODEL ANSWER:

AAPT stands for Android Asset Packaging Tool.

It is a tool used for processing Android application resources and packaging them for the application.

IMPORTANCE OF AAPT:

1. It processes XML resource files.

2. It processes application resources such as images.

3. It generates resource identifiers.

4. It compiles resources required by the application.

5. It helps package application resources.

Thus, AAPT plays an important role in processing and packaging resources during Android application development.

==========

Q7. (5 MARKS)

What is the app package format?

==========

MODEL ANSWER:

The application package format used by Android is called APK.

APK stands for Android Package.

An APK is the package file used to distribute and install an

Android application.

An APK generally contains:

1. Android application code.
2. DEX files.
3. AndroidManifest.xml.
4. Application resources.
5. Assets.
6. Compiled resources.

The basic process is:

Android Application

↓

APK

↓

Installation on Android Device

Therefore, APK is the package format used for Android applications.

==========

Q8. (5 MARKS)

What is the purpose of the Android Manifest file

(AndroidManifest.xml)?

==========

MODEL ANSWER:

AndroidManifest.xml is an important configuration file of an Android application.

It provides essential information about the application to the Android operating system.

PURPOSES:

1. It declares application components such as Activities, Services and Broadcast Receivers.

2. It specifies permissions required by the application.

3. It provides application-related information.

4. It can declare required features and capabilities.

5. It provides information needed by the Android system to run the application.

Therefore, AndroidManifest.xml acts as an important configuration file connecting the application with the Android system.

==========

Q9. (5 MARKS)

Define an Activity.

==========

MODEL ANSWER:

An Activity is an Android application component that represents a single screen with a user interface.

It is responsible for providing an interface through which the user can interact with the application.

For example, a shopping application may have:

Login Screen → Login Activity

Home Screen → Home Activity

Product Screen → Product Activity

Payment Screen → Payment Activity

An Activity also has a lifecycle consisting of different callback methods such as onCreate(), onStart(), onResume(), onPause(), onStop() and onDestroy().

Thus, Activity is one of the fundamental components of an Android application.

==========

Q10. (5 MARKS)

The full form of ADB is:

==========

MODEL ANSWER:

ADB stands for Android Debug Bridge.

It is a command-line tool that allows communication between a computer and an Android device or emulator.

ADB can be used for:

1. Installing applications.
2. Debugging applications.
3. Accessing the device shell.
4. Transferring files.
5. Running commands on an Android device.

Therefore, ADB is an important tool used during Android application development and debugging.

==========

Q11. (5 MARKS)

What is Gradle?

==========

MODEL ANSWER:

Gradle is a build automation tool used in Android Studio.

It is responsible for managing the build process of an Android application.

FUNCTIONS OF GRADLE:

1. Compiles application code.

2. Manages project dependencies.

3. Manages build configurations.

4. Creates APK files.

5. Supports different build types and configurations.

For example, when an Android application is built, Gradle compiles the code, processes resources and packages the application.

Therefore:

GRADLE = BUILD + DEPENDENCY + PROJECT MANAGEMENT

==========

Q12. (10 MARKS)

What do you mean by Activity Life Cycle in Android?

Which are the core callback methods used there? Draw and explain the different phases with a neat diagram. Also include the relevant real-time scenario for each.

==========

MODEL ANSWER:

The Activity Life Cycle describes the different states through which an Android Activity passes from creation to destruction.

The main callback methods are:

onCreate()

↓

onStart()

↓

onResume()

↓

RUNNING

↓

onPause()

↓

onStop()

↓

onDestroy()

1. onCreate()

This method is called when the Activity is first created.

It is generally used to:

- Initialize the Activity.
- Set the layout.
- Initialize required variables.

REAL-TIME SCENARIO:

When a user opens a shopping application, the home Activity is created.

2. onStart()

This method is called when the Activity becomes visible to the user.

REAL-TIME SCENARIO:

The shopping application's home screen becomes visible on the mobile screen.

3. onResume()

This method is called when the Activity comes to the foreground and the user can interact with it.

REAL-TIME SCENARIO:
The user starts scrolling through products and interacting with the shopping application.

4. onPause()

This method is called when the Activity loses focus or is partially covered by another screen.

REAL-TIME SCENARIO:
A notification, dialog or another temporary interface appears over the current Activity.

5. onStop()

This method is called when the Activity is no longer visible to the user.

REAL-TIME SCENARIO:
The user opens another application and the shopping Activity is completely hidden.

6. onDestroy()

This method is called when the Activity is being destroyed.

REAL-TIME SCENARIO:

The Activity is closed or removed from memory.

LIFE CYCLE SUMMARY:

onCreate() → Activity is created

onStart() → Activity becomes visible

onResume() → User can interact

onPause() → Activity loses focus

onStop() → Activity is no longer visible

onDestroy() → Activity is destroyed

CONCLUSION:

The Activity Life Cycle helps Android manage application resources and control what happens when an Activity is created, displayed, paused, stopped or destroyed.

==========

Q13. (5 MARKS)

What is the default orientation?

==========

MODEL ANSWER:

The default orientation of a LinearLayout is horizontal.

In horizontal orientation, child views are arranged from left to right.

Example:

View 1 | View 2 | View 3

The orientation can be changed using:

android:orientation="vertical"

or

android:orientation="horizontal"

In vertical orientation, the views are arranged from top to bottom.

Example:

View 1

View 2

View 3

==========

Q14. (5 MARKS)

How does android:layout_weight work?

==========

MODEL ANSWER:

android:layout_weight is used in LinearLayout to determine how the available space is distributed among child views.

It specifies the relative share of extra space that a view should receive.

Example:

View 1 → weight = 1

View 2 → weight = 1

Both views receive equal weighted space.

If:

View 1 → weight = 1

View 2 → weight = 2

View 2 receives twice the weighted space of View 1.

Therefore:

layout_weight = Share of available space

It is useful when creating flexible layouts that adjust to

different screen sizes.

==========

Q15. (5 MARKS)

What is android:weightSum used for?

==========

MODEL ANSWER:

android:weightSum is used to specify the total weight available

in a LinearLayout.

For example:

android:weightSum="3"

If three child views have:

View 1 → weight = 1

View 2 → weight = 1

View 3 → weight = 1

Total weight = 1 + 1 + 1 = 3

Therefore, each view receives approximately one-third of the available weighted space.

In simple words:

weightSum = Total weight available

layout_weight = Individual view's share

==========

Q16. (5 MARKS)

What is the difference between gravity and layout_gravity?

==========

MODEL ANSWER:

The main difference is what they control.

GRAVITY:

gravity controls the position of the content inside a View.

For example, in a TextView:

android:gravity="center"

This places the text in the center of the TextView.

LAYOUT_GRAVITY:

layout_gravity controls the position of the View inside its parent layout.

For example:

android:layout_gravity="center"

This positions the TextView itself in the center of its parent.

DIFFERENCE:

gravity
→ Controls content inside the View.

layout_gravity
→ Controls the View inside its parent.

EASY WAY TO REMEMBER:

gravity → CONTENT

layout_gravity → VIEW

==========

Q17. (10 MARKS)

Compare the different Layouts.

==========

MODEL ANSWER:

Android provides different layouts to arrange views and create user interfaces.

The important layouts are:

1. LINEAR LAYOUT

LinearLayout arranges child views in a single direction.

The direction can be:

- Horizontal
- Vertical

Example:

View 1

View 2

View 3

It is useful for simple row or column-based interfaces.

2. RELATIVE LAYOUT

RelativeLayout positions views relative to:

- Other views
- The parent layout

For example, one TextView can be placed below another TextView.

It provides flexible positioning of views.

3. CONSTRAINT LAYOUT

ConstraintLayout positions views using constraints.

A view can be constrained to:

- Parent
- Other views
- Guidelines

It is useful for creating complex and responsive interfaces with a flatter view hierarchy.

4. FRAME LAYOUT

FrameLayout is generally used to place views on top of each other.

For example, an ImageView can be used as a background and a TextView can be placed over it.

It is useful when overlapping views are required.

5. TABLE LAYOUT

TableLayout arranges views in rows and columns.

It is useful for displaying information in a table-like structure.

COMPARISON:

Layout	Main Purpose
LinearLayout	Arranges views in a row or column.

RelativeLayout Positions views relative to other views or the parent.

ConstraintLayout Uses constraints to position views and create responsive interfaces.

FrameLayout Allows views to overlap each other.

TableLayout Arranges views in rows and columns.

CONCLUSION:

The choice of layout depends on the user interface requirement. LinearLayout is suitable for simple arrangements, while ConstraintLayout is useful for more complex and responsive interfaces.

==========

Q18. (10 MARKS)

What are the Performance Pitfalls (Double Taxation) of Linear Layout? Explain.

==========

MODEL ANSWER:

Double taxation is a performance issue that can occur in LinearLayout when the layout needs to measure its child views more than once.

This is particularly associated with the use of layout weights.

WHY DOES IT HAPPEN?

When weights are used, Android may need to perform additional measurement calculations to determine how the available space should be distributed among the child views.

Therefore, some child views may be measured more than once.

EXAMPLE:

Suppose a LinearLayout contains several child views:

```
LinearLayout
  |
  |--- View 1 (weight = 1)
  |--- View 2 (weight = 1)
  |--- View 3 (weight = 1)
```

Android needs to determine how the available space should be distributed according to their weights.

This may require additional measurement passes.

EFFECT ON PERFORMANCE:

Double taxation can result in:

1. Additional measurement operations.

2. More processing work.

3. Increased layout time.

4. Reduced performance in complex or deeply nested layouts.

WHY IS IT A PROBLEM?

For simple layouts, the effect may be small.

However, if many LinearLayouts are nested inside one another
and weights are heavily used, repeated measurements can increase
the cost of rendering the user interface.

HOW TO REDUCE THE PROBLEM:

1. Avoid unnecessary nested LinearLayouts.

2. Avoid excessive use of layout weights when they are not required.

3. Use a suitable layout structure.

4. Use ConstraintLayout where appropriate to create a flatter hierarchy.

CONCLUSION:

Double taxation refers to additional measurement/layout work that may occur in LinearLayout, especially when weights are used. Excessive use of nested LinearLayouts and weights can affect UI performance, so layouts should be designed efficiently.